

Synchronization

Physical Clocks, Logical Clocks

Distributed Systems Synchronization

1 Synchronous Systems

- Physical Clocks
 - Quartz Clocks
 - Atomic Clocks
 - GPS
- Network Time Protocol
- Totally Ordered Multicast

2 Asynchronous Systems

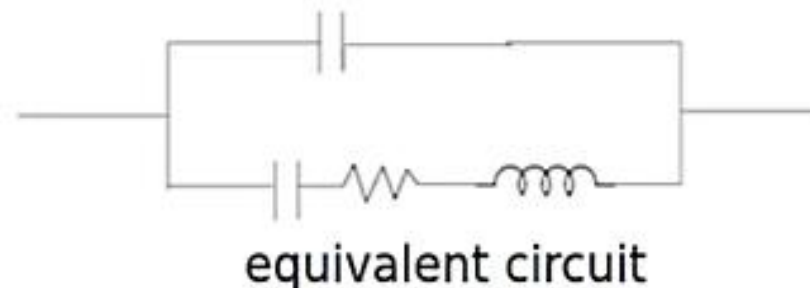
- Happens Before Relationship
- Totally Ordered Mutual Exclusion

Synchronous Systems

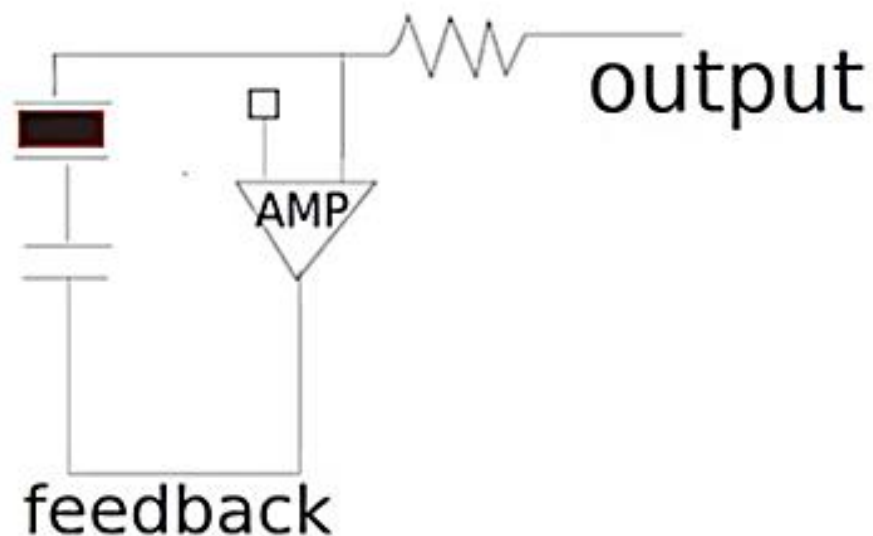
Quartz Based Clock

Quartz Oscillator

- Computers clock use a quartz crystal to generate a clock signal.
- Quartz is a piezoelectric material – generates a voltage, when subjected to mechanical stress.
- Resistant to temperature fluctuations.



Quartz Clock II



- The quartz oscillator is a part of a self feedback loop.
- It typically oscillates at 32 KHz.
- Processors generate a higher frequency by dividing this clock.
- The clock drift is ± 15 seconds per month (6 ppm).
- A regular quartz clock is not suitable for large distributed systems.

Clock Skew Problem

Clock skew(offset): the difference between the times on two clocks

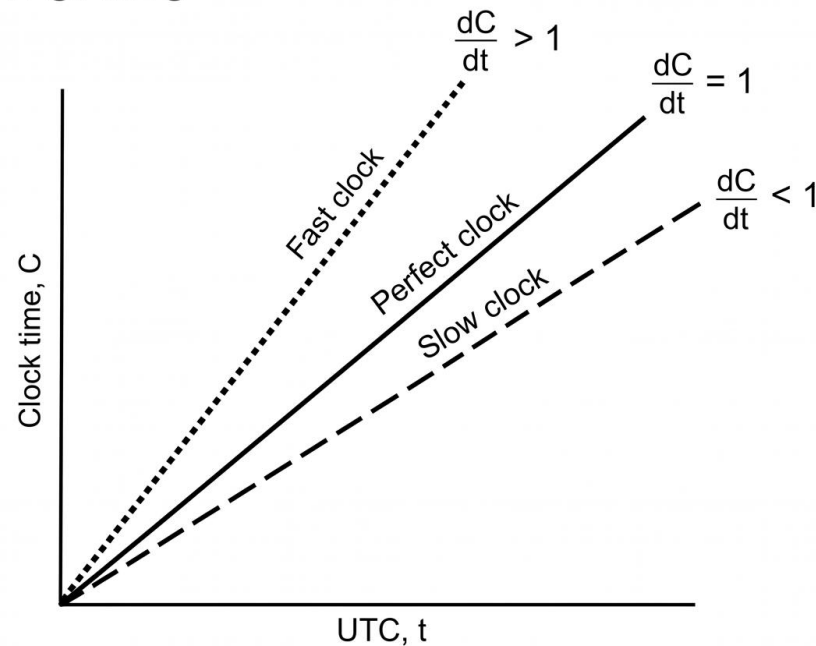
Clock drift : they count time at different rates

Ordinary quartz clocks drift by $\sim 1\text{sec}$ in 11-12 days. (10^{-6} secs/sec).

High precision quartz clocks drift rate is $\sim 10^{-7}$ or 10^{-8} secs/sec

Physical clock drift rate

- Maximum drift rate
 - One can determine how often they should be synchronize



Not all clock's tick precisely at the current rate.

Atomic Clock

Atomic Clock

- Uses a Caesium-133 atom as an oscillator.
- Uses a similar feedback based circuit as the quartz clock.
- Accuracy : 10^{-8} ppm

Use of Atomic Clock: GPS



- Each satellite broadcasts its position (x_i, y_i, z_i) and time t_i
- The time is obtained through an atomic clock.

Finding the Position through GPS

- Current position: (x, y, z)
- The drift between the receiver clock and the atomic clocks is d .
- The time at which the receiver receives the message is t_r .
- Setup equation:

$$\sqrt{(x - x_i)^2 + (y - y_i)^2 + (z - z_i)^2} = (t_r - t_i + d) \times c$$

- c is the speed of light
- For four unknowns x, y, z, d , we need at least four equations

Network Time Protocol

- There are a set of network time servers that have accurate clocks (stratum 1).
- These servers might in turn synchronize with servers that have even more accurate clocks (stratum 0).
- A client machine needs to contact a NTP time server and find the drift between the clocks.
- There are different [clock synchronization](#) algorithms.

Cristian's Algorithm

- 1 Client sends a request to the server at its local time t_1 .
- 2 Server receives it at its local time t_2 .
- 3 Server sends a reply at its local time t_3 .
- 4 Client receives the replay at t_4 .

Calculating the Drift - Δ

If we assume that the jitter in the network is 0, then the request and response take the same amount of time. We have

$$\begin{aligned} t_2 - (t_1 + \Delta) &= t_4 + \Delta - t_3 \\ \Rightarrow \Delta &= \frac{(t_2 - t_1) + (t_3 - t_4)}{2} \end{aligned} \quad (1)$$

Berkeley Algorithm

- A master is chosen by some method among a group of nodes.
- The master uses Cristian's algorithm to find the clock drift with each slave.
- The master computes the mean value of the drift.
- The master sends an update to each slave regarding the amount that the slave needs to shift its clock.
- This ensures that the clocks of most slaves are relatively synchronized with each other.
- The algorithm also aims to minimize the amount by which each slave needs to adjust its clock.

External vs Internal Synchronization

- Consider a group of processes
 - **External synchronization**
 - Each process $C(i)$'s clock is within a bounded D of a well-known clock S external to the group
 - $|C(i) - S| < D$ at all times.
 - External clock may be connected to UTC (Universal Coordinated Time) or an atomic clock.
 - **Example: Christian's algorithm, NTP**
 - **Internal Synchronization**
 - Every pair of processes in group have clocks within bound D
 - $|C(i) - C(j)| < D$ at all times and for all processes i, j .
 - **Example: Berkley Algorithm, DTP**
-

Totally Order Multicast with Synchronized Clocks

Problem

Nodes randomly send messages to a subset of other nodes. The network has a non-deterministic delay. It is bounded by Δ . Ensure that all the messages are delivered in the same order at all nodes.

Solution

Sender: Timestamp every message with the local time.

Receiver:

- 1 For a message with timestamp t , transfer it to the receive queue at time $t + \Delta$.
- 2 Deliver the messages in the receive queue in the order of their timestamps.

Totally Order Multicast with Synchronized Clocks

Problem

Missing messages from a process P can violate delivery order.



- (*) missing a message
- (*) exactly the same order

Core Concept

Deliver a message only when it is known that

- 1 All messages with a smaller timestamp are already present in the receive queue.
- 2 No message with a smaller timestamp will ever arrive.

Asynchronous Systems

Why do we need global clocks?

- For causally ordering events in a distributed system
 - **Example:**
 - Transaction T transfers Rs 10,000 from S1 to S2
 - Consider the situation when:
 - State of S1 is recorded after the deduction and state of S2 is recorded before the addition
 - State of S1 is recorded before the deduction and state of S2 is recorded after the addition
- Should not be confused with the clock-synchronization problem



What data is being transmitted? 0101?



Yes, if this is the clock



If this is the clock, then 01110001

The receiver needs to know the clock of the sender

Distributed Process Synchronization

- Communication between processes in a distributed system can have unpredictable delays, processes can fail, messages may be lost.
- Synchronization in distributed systems is harder than in centralized systems because the need for distributed algorithms.

Properties of distributed algorithms:

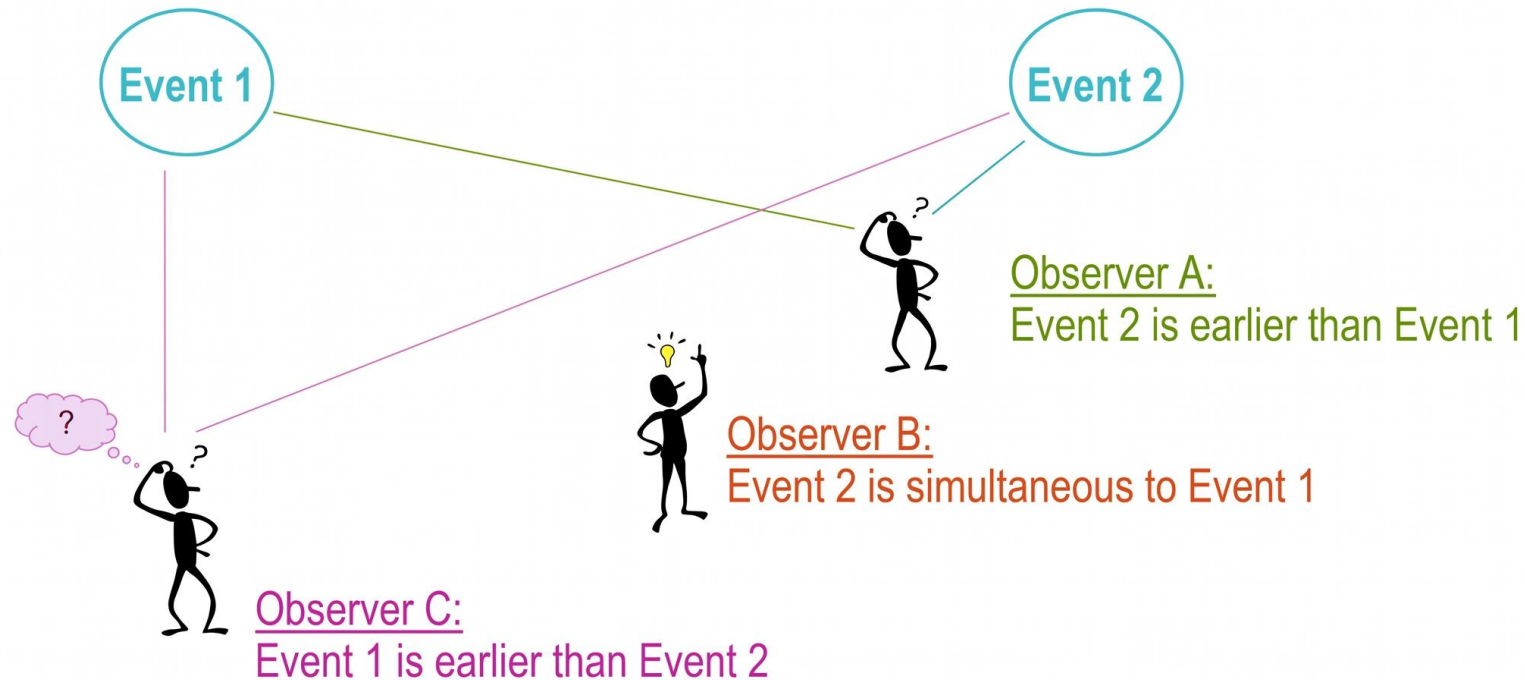
- ✓ The relevant information is scattered among multiple machines.
- ✓ Processes make decisions based only on locally available information.
- ✓ A single point of failure in the system should be avoided.
- ✓ No common clock or other precise global time source exists.

Challenge:

How to design schemes so that multiple systems can coordinate or synchronize to solve problems efficiently?

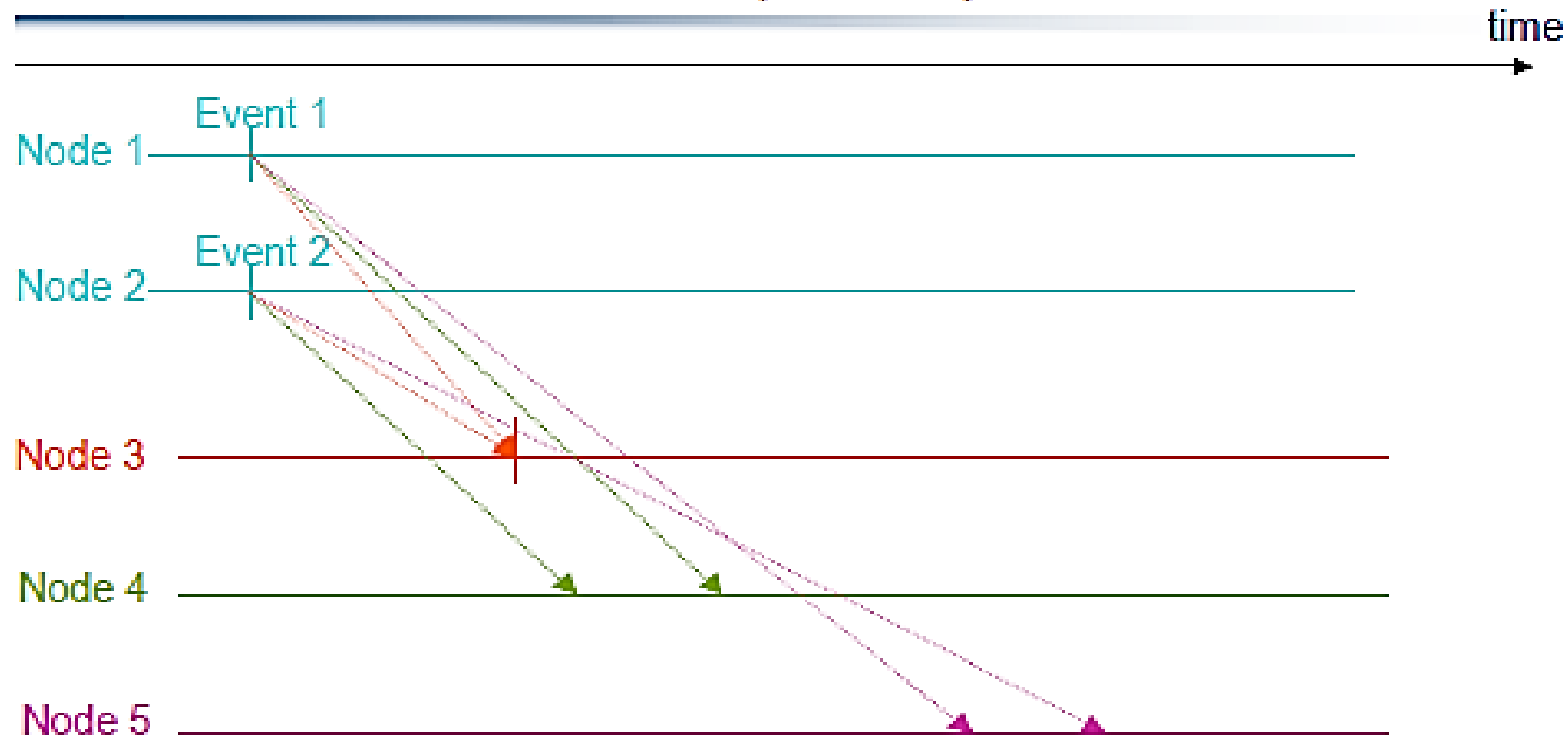
The Myth of Simultaneity

“Event 1 and event 2 at same time”



$$e1 \parallel e2 = e1 \rightarrow e2 \mid e2 \rightarrow e1$$

Event Timelines (Example of previous Slide)



Note: The arrows start from an event and end at an observation.
The slope of the arrows depend of relative speed of propagation

Absence of global clock

No system-wide common clock (global clock)

Reasons:

Suppose a global clock exists. Two different processes can observe the clock at different times due to unpredictable message delays. Hence two different instances in physical time may be perceived as a single instance in physical time.

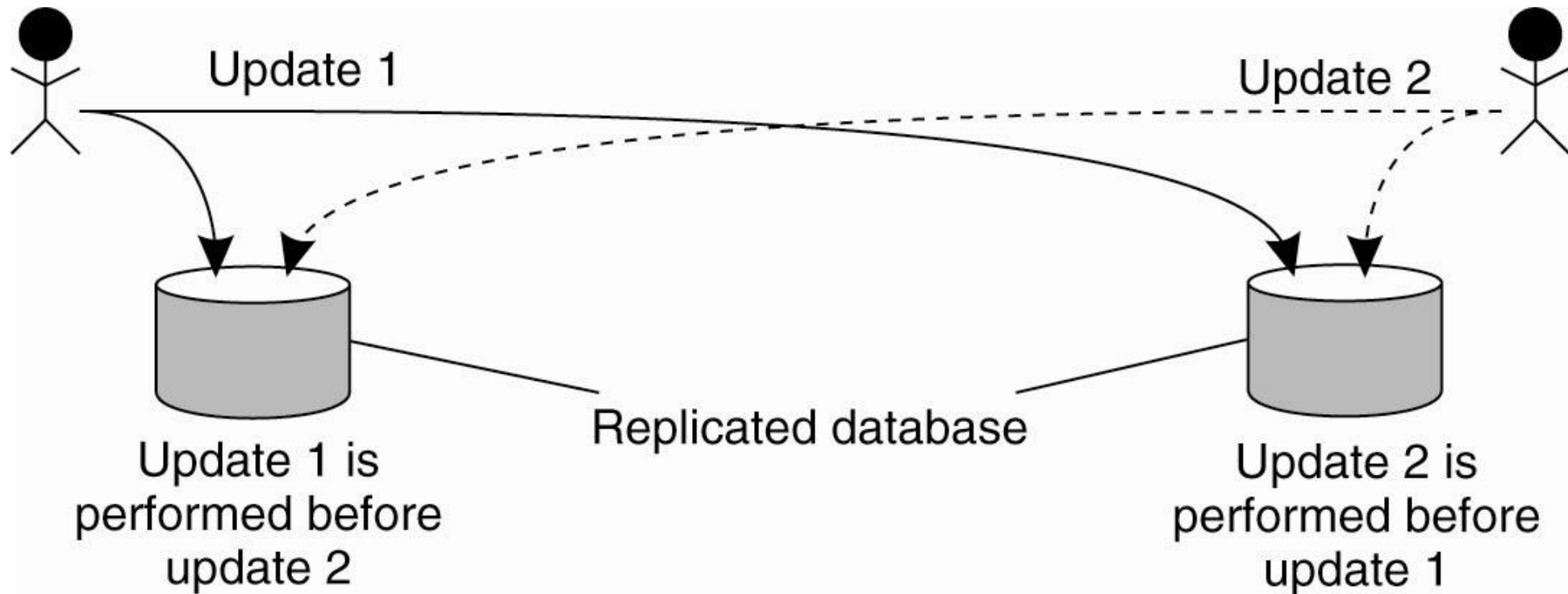
If each process has a local clock In a distributed system, each node has its own clock and each one of those clocks may run at a different rate or granularity due to technological limitations, which means they will drift apart from each other.

Hence two different instances in physical time may be perceived as a single instance in physical time.

Impact of the absence of global time

- Scheduling of processes by an OS depends on the temporal order of process arrival.
- Difficult to design and debug distributed programs since it is difficult to reason about temporal order of events in distributed systems.
- Without a global clock, difficult to collect up-to-date information on the state of the entire system.

Updating a replicated database and leaving it in an inconsistent state



Why need to synchronize clocks?

